



## TTK4147 Real-time Systems

### 06 Priority Inversion

24.09.2025

1

TTK4147 – 2025: Priority Inversion

 NTNU

1

### Today

- Priority Inversion
- Priority Inheritance
- Priority Ceiling
- Worst Case Blocking Time with Inheritance

2

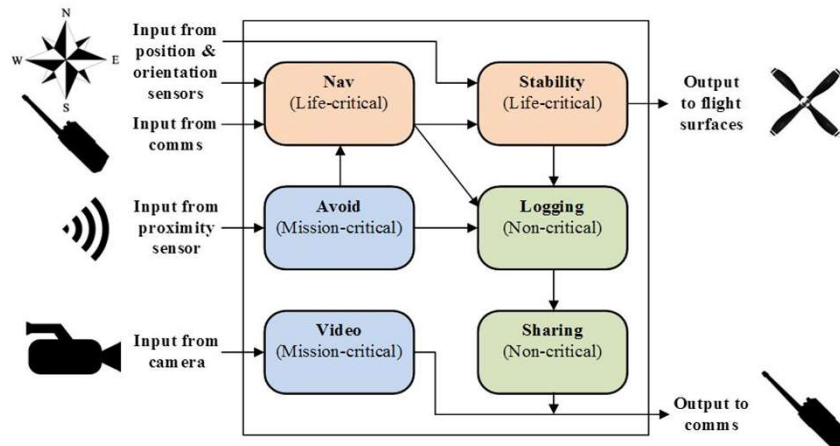
TTK4147 – 2025: Priority Inversion

 NTNU

2

### Given a mixed Criticality Unmanned Aerial Vehicle (UAV)

Tasks of different criticality; Safety critical, Mission critical, Non critical



3

TTK4147 – 2025: Priority Inversion

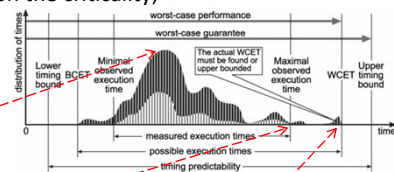
NTNU

3

### Tripartite mixed Criticality, UAV Example

Tasks are given different **priority** depending on the criticality;

- Safety critical, Highest priority
- Mission critical: Medium priority
- Non critical: Low priority
- Background tasks: Lowest priority



| Criticality | Mean Execution Time | Max Observed WCET | Max Theoretical WCET | Priority |
|-------------|---------------------|-------------------|----------------------|----------|
| Safety      | 50%                 | 60%               | 90%                  | 3        |
| Mission     | 30%                 | 40%               | (50%)                | 2        |
| Non         | 20%                 | (30%)             | (40%)                | 1        |
| Total       | 100%                | 100%              | 90%                  |          |

- If all tasks perform at mean execution time, you will even be able to do the non-critical tasks.
- The background jobs is meant to fill idle time and may be of unlimited extent.



What about garbage collection in FreeRTOS? When to execute?

4

TTK4147 – 2025: Priority Inversion

NTNU

4

## What blocking (?) of high priority tasks may result in

| Criticality | Mean Execution Time | Max Observed WCET | Max Theoretical WCET | Priority |
|-------------|---------------------|-------------------|----------------------|----------|
| Safety      | 50%                 | 60%               | 90%                  | 3        |
| Mission     | 30%                 | 40%               | (50%)                | 2        |
| Non         | 20%                 | (30%)             | (40%)                | 1        |
| Total       | 100%                | 100%              | 90%                  |          |

An inversion of the priority may happen

- Priority inversion is when a high priority task has to wait for a lower-priority task.

5

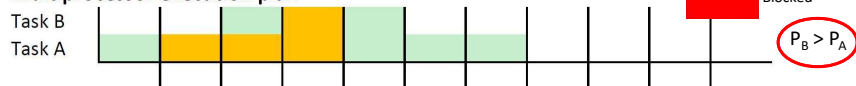
TTK4147 – 2025: Priority Inversion



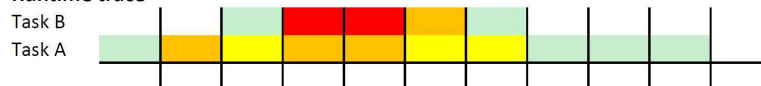
## Priority Inversion

Typical example of priority inversion is:

### Multiprocessor execution plan



### Runtime trace



- A low-priority task locks a mutex, which will remain locked while it access a shared resource.
- A high-priority task need the same shared resource, and waits for the mutex.

**This is actually acceptable**

6

TTK4147 – 2025: Priority Inversion

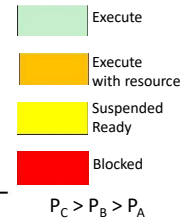
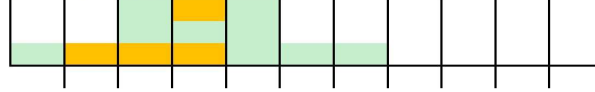


## Priority Inversion

- But, a more sophisticated effect is that the high-priority task, in practice, get the low-priority.

### Multiprocessor execution plan

Task C  
Task B  
Task A



### Runtime trace

Task C  
Task B  
Task A



- If you have a *medium-priority* task, then it can prevent the low-priority task from completing its work and returning the mutex.
- This will prevent the high-priority task from running.

**This is NOT acceptable**

7

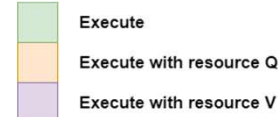
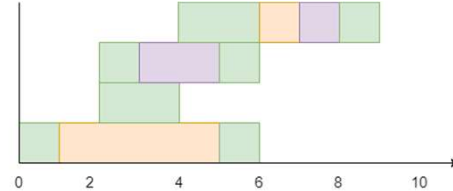
TTK4147 – 2025: Priority Inversion

NTNU

## Priority Inversion

### Multiprocessor execution plan:

Task D (4)  
Task C (3)  
Task B (2)  
Task A (1)



### No Priority Inheritance

Task D (4)  
Task C (3)  
Task B (2)  
Task A (1)



Task D has to wait extra for Task C,B to finish

8

TTK4147 – 2025: Priority Inversion

NTNU

## Task Interactions and Blocking, definitions

### Fundamental

- If a task is suspended waiting for a lower-priority task to complete some required computation, then **the priority model** is, in some sense, being undermined
- It is said to suffer **priority inversion**
- If a task is waiting for a lower-priority task, it is said to be **blocked**

9

TTK4147 – 2025: Priority Inversion

NTNU

9

## The problem has been known for long: A much publicized example is The Mars Path-finder Incident

- A problem due to priority inversion nearly caused the lose of the Mars Path-finder mission.
- As a **Shared resource** bus got heavily loaded, critical data was not transferred.
- Time-out on this data was used as an indication of failure and lead to re-boot (watchdog).
- Solution was a patch that turned on *priority inheritance*, which solved the problem.



Sojourner

Fun Facts:  
 Launch: 1996-12-04  
 Price: 184 mill USD  
 Weight 10,5 kg  
 Speed: 0,024 km/h  
 Mission: 85 days

(What is priority inheritance?)



10

TTK4147 – 2025: Priority Inversion

NTNU

10

## Priority Inheritance

With priority inheritance, a task's priority is no longer static. If a task **p** is suspended waiting for task **q** to undertake some computation, then the priority of **q** becomes equal to the priority of **p** (if it was lower to start with).

The priority inheritance may lead to dead locks as the low priority task with inherited high priority may request for a resource already locked by the blocked high priority task



What could be the solution for this?

Define an order in which the resources must be requested.  
(Prevents circular wait)

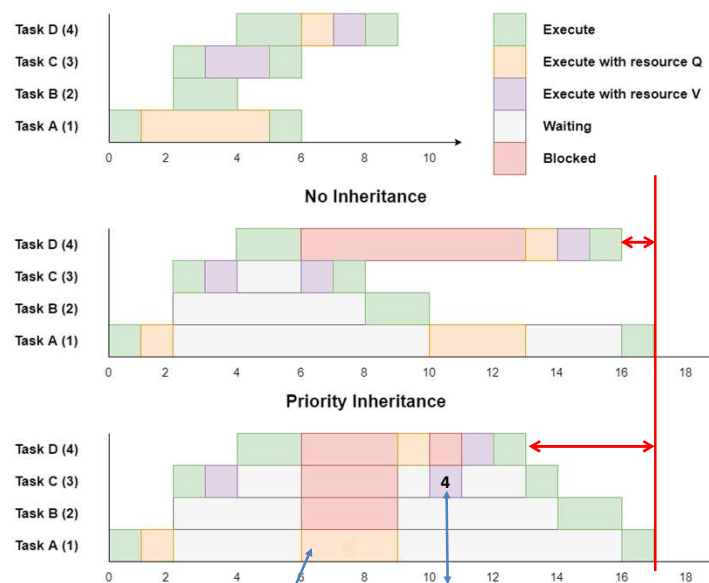
11

TTK4147 – 2025: Priority Inversion



11

## Priority Inheritance example



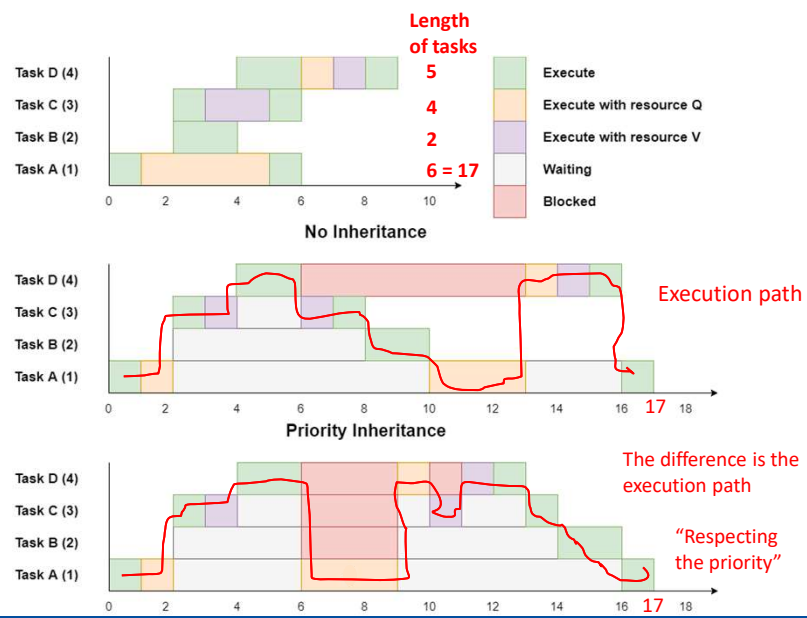
Here, task A (C) inherit the priority of the blocked task D, but D finished before B&C

12

TTK4147 – 2025: Priority Inversion



## Fundamental CPU allocation, one CPU / core



13

TTK4147 – 2025: Priority Inversion

NTNU

## Multi step Priority Inheritance

### Multiprocessor execution plan



### Runtime trace



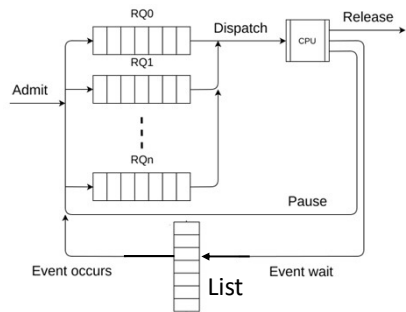
14

TTK4147 – 2025: Priority Inversion

NTNU

## Preemptive Priority Based Scheduling

- Each priority level gets its own ready queue, and the dispatcher will at each tick check higher priority queues first.
- If no task with higher priority than running task, the current task will continue.
- Lower priority tasks might suffer from "starvation".



Just for fun; what does "List" symbolize? Which task state?

Blocked state

15

TTK4147 – 2025: Priority Inversion

NTNU

15

## Priority Ceiling Protocols

- Priority inheritance reduce the effect of priority inversion of high-priority tasks.
  - Is it possible to reduce it further?
  - What about deadlocks?
- A low priority task that access a shared resource does not get a priority inheritance effect before a high-priority task tries to access the same resource.
- Would it be better to immediately increase the priority so it will complete its use of the shared resource as fast as possible?
- We will cover two priority ceiling protocols:
  - Original ceiling priority protocol.
  - Immediate ceiling priority protocol.



16

TTK4147 – 2025: Priority Inversion

NTNU

16



## Priority Ceiling Protocols on a Single Processor

### Statements:

- A high-priority task can be blocked at most once during its execution by lower-priority tasks
- Deadlocks are prevented
- Transitive\* blocking is prevented
- Mutual exclusive access to resources is ensured (by the protocol itself)

\*Transitive blocking, chains of blocks, task c being blocked by task b which is blocked by task a and so on

17

TTK4147 – 2025: Priority Inversion



17

## Immediate Ceiling Priority Protocol (ICPP)

- Each task has a static default priority assigned (perhaps by the deadline monotonic scheme)
- Each shared resource is assigned the priority of the highest priority task that will use it.
- When a task is allowed to access a shared resource, it will inherit its priority. A task got the priority max of its own static and the ceiling priority of any **resources** it has locked.
- A task can only lock a resource if the task's priority is **higher than** the ceiling of any **currently locked resource** (excluding any that it has already locked itself).
  - This means that when a task starts using a resource, it is guaranteed to be uninterrupted until it is done with the resource.  
(Prevent that the task is holding a resource while being not running, potential deadlock)
  - Minimize the time a resource is *locked*.

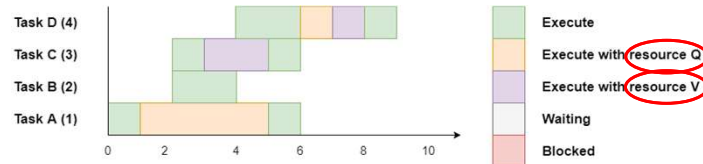
18

TTK4147 – 2025: Priority Inversion

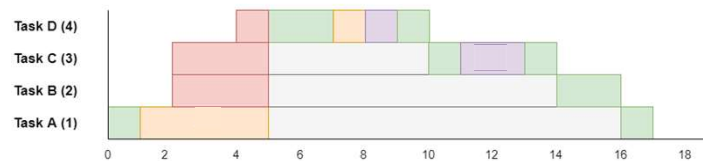


18

## Immediate Ceiling Priority Protocol (ICPP)



- Each shared resource is assigned the priority of the highest priority task that use it.  
 $P_Q = 4, P_V = 4$
- A task got the priority max of its own static and the ceiling priority of any resources it **has locked**



19

TTK4147 – 2025: Priority Inversion

NTNU

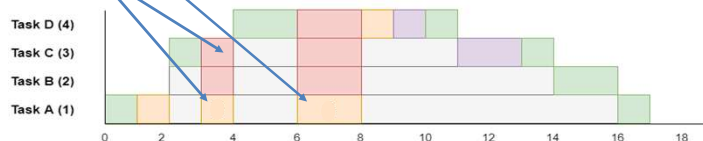
19

## Original Ceiling Priority Protocol (OCP)



- Each task has a static default priority assigned.
- Each resource has a static ceiling value defined equal to the maximum priority of the tasks that will use it.  
 $P_V: 4$   
 $P_Q: 4$
- A task has a dynamic priority which is the maximum of its own static priority and that of any higher priority **tasks** it blocks (thus, it will not be preempted by another task asking for this resource).
- A task can only lock a resource if the task's priority is **higher than** the ceiling of any **currently locked resource** (excluding any that it has already locked itself).  
 $P_C = 3$ , not higher than  $P_Q = 4$

Original Ceiling Priority Protocol (OCP)



20

TTK4147 – 2025: Priority Inversion

NTNU

20

## Original Ceiling Priority Protocol (OCP) cont.

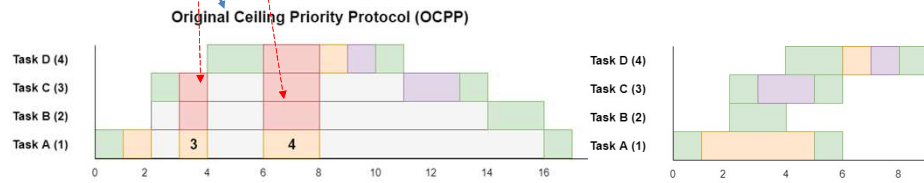
What about

"A high-priority task can be blocked at most once during its execution by lower-priority tasks"?

From Burns-Wellings, 11.9 page 387:

*The benefit of the ceiling protocol is that a high-priority task can only be blocked once (per activation) by any lower-priority task.*

*The priority ceiling protocols ensure that a task is only blocked once during each invocation. Figure 11.8, however, appears to show task B (and task C) suffering two blocks. What is actually happening, is that a single block is being broken in two by the preemption of task D.*



21

TTK4147 – 2025: Priority Inversion

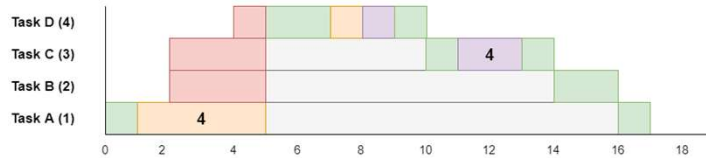
NTNU

21

## OCP vs ICPP 1 av 3



**Immediate Ceiling Priority Protocol (ICPP)**



**Original Ceiling Priority Protocol (OCP)**



22

TTK4147 – 2025: Priority Inversion

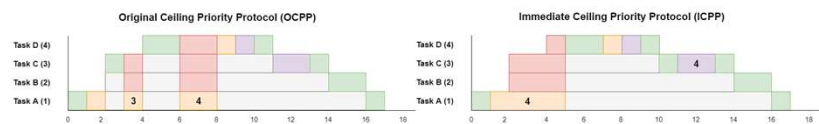
NTNU

22

## OCPP vs ICPP 2 av 3

Although the worst-case behavior of the two ceiling schemes is identical (from a scheduling point of view), there are some points of difference:

- ICPP is easier to implement than the original (OCPP), which must keep track of what is blocking what.
- ICPP leads to less context switches as blocking is prior to first / start execution
- ICPP requires more priority movements as this happens with all resource usage



23

TTK4147 – 2025: Priority Inversion

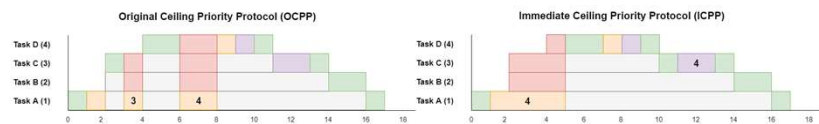


23

## OCPP vs ICPP 3 av 3

- OCPP changes priority only if an actual block has occurred

Note that ICPP is called Priority Protect Protocol in POSIX and Priority Ceiling Emulation in Real-Time Java



24

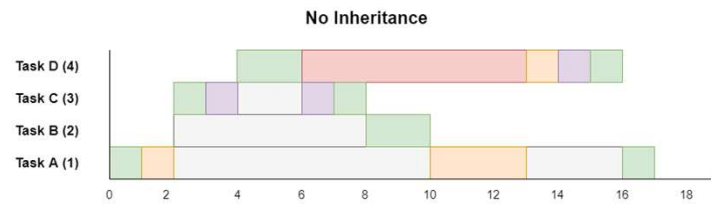
TTK4147 – 2025: Priority Inversion



24

## Priority Inversion Blocking Time

- The general message is that priority inversion could cause higher priority tasks to be delayed by lower priority ones.
  - Priority inheritance and ceiling can reduce this problem.
- But in real-time computing, you of course know that there is one thing that is more important than speed...



25

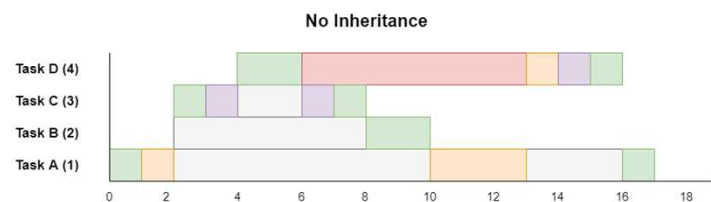
TTK4147 – 2025: Priority Inversion

NTNU

25

## Priority Inversion Blocking Time

- Predictability!!!**
  - The problem with the example below is not that task D is blocked.
  - Its that the worst case blocking time is really bad.
- With priority inheritance, it is easier to calculate the worst case blocking time.



26

TTK4147 – 2025: Priority Inversion

NTNU

26

## Worst Case Blocking Time with Inheritance

- A task can only be blocked once due to each shared resource.
  - In example task D is first blocked due to Q then due to V.
- How long will a higher priority task be blocked due to a shared resource?
  - Block cause low priority task to get high priority.
  - Max block time is the max time the low priority task use the resource and it is blocking.

$$B_i = \sum_{k=1}^K usage(k, i) C(k)$$



27

TTK4147 – 2025: Priority Inversion



27

## Adding Block Time to Response Time Analysis 1 of 2

Response-time: The time from a task starts execution to it is completed [Burns and Wellings]

Previously learned:

$$WCET: R_i = C_i + I_i$$

Block time:

- The *usage* function:
  - 1 for shared resources that are used by lower and higher or equal (than i) priority tasks.
  - 0 for others.
- $C(k)$  is the WCET (the length of time) of the k critical section / k resource (i.e. when the blocking occurs)

$$B_i = \sum_{k=1}^K usage(k, i) C(k)$$

28

TTK4147 – 2025: Priority Inversion



28

## Adding Block Time to Response Time Analysis 2 of 2

$$B_i = \sum_{k=1}^K usage(k,i)C(k)$$

NB Different

$$R_i = C_i + B_i + I_i$$

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

- The worst case blocking time can be added to the response (turnaround) time expression.

- Can then be used in response (turnaround) time analysis.

29

TTK4147 – 2025: Priority Inversion

NTNU

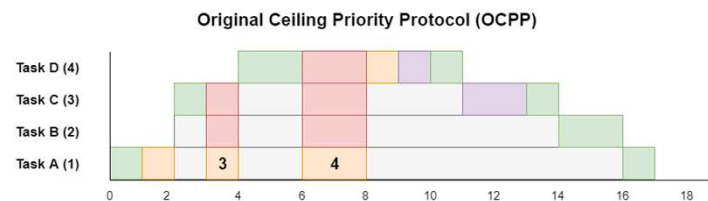
29

## Worst Case Blocking Time with Ceiling

- When using priority ceiling, a high-priority task can only be blocked ONCE by a lower priority task.
  - Does not matter how many different shared resources or lower-priority tasks that exist.
  - The worst case blocking time is the same for OCPP and ICPP.
  - Compared to priority inheritance, more tasks will be blocked, but max one time each.

$$B_i = \max_{k=1}^k usage(k,i)C(k)$$

(To be used in  $R_i = C_i + B_i + I_i$ )



30

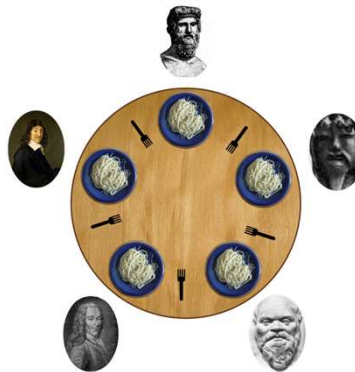
TTK4147 – 2025: Priority Inversion

NTNU

30

## The dining philosophers' problem

- Five silent philosophers sit at a round table with bowls of spaghetti.
- Forks are placed between each pair of adjacent philosophers.
- Each philosopher must alternately think and eat.
- A philosopher can only eat spaghetti when they have both left and right forks.
- After an individual philosopher finishes eating, they need to put down both forks so that the forks become available to others.
- A philosopher can take the fork on their right or the one on their left as they become available, but cannot start eating before getting both forks



Do we have a risk of deadlock? In case, how?

If all philosophers take one fork before anyone try to take a second fork

31

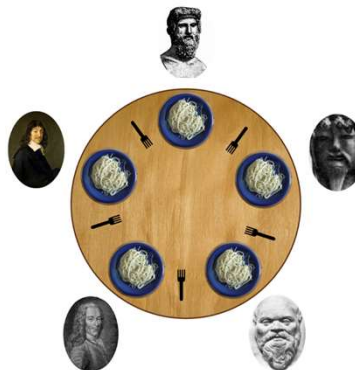
TTK4147 – 2025: Priority Inversion

NTNU

31

## The dining philosophers' problem

- In the following, the problem will be regarded with a single CPU as view and using priority ceiling protocols.
- Remember the following statements:
  - A high-priority task can be blocked at most once during its execution by lower-priority tasks
  - Deadlocks are prevented
  - Transitive blocking is prevented
  - Mutual exclusive access to resources is ensured (by the ceiling protocol itself)



32

TTK4147 – 2025: Priority Inversion

NTNU

32

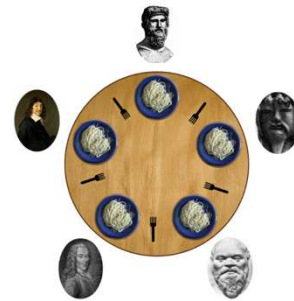


## Priority Ceiling and Deadlocks, OCPP

- Priority ceiling effectively prevent deadlocks from "dining philosophers" problems.

With OCPP:

- A philosopher is only allowed to take a fork, if no other fork is taken.
- When he eventually is able to take the fork, no other philosopher will be allowed to take the other one, but the lucky one is allowed to take the other fork.



Because:

- Each task has a static default priority assigned.
- Each resource has a static ceiling value defined equal to the maximum priority of the tasks that will use it.
- A task has a dynamic priority which is the maximum of its own static priority and that of any higher priority tasks it blocks.
- A task can only lock a resource if the task's dynamic priority is **higher than the ceiling** of any currently locked resource (excluding any that it has already locked itself).

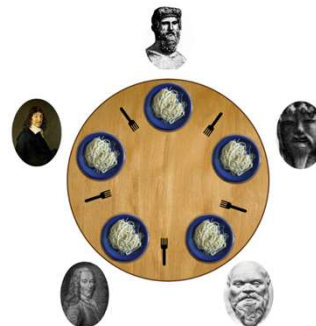
33

TTK4147 – 2025: Priority Inversion



## Priority Ceiling and Deadlocks, ICPP

- With ICPP:
  - A philosopher only try to take a fork if both are free
  - When a philosopher takes one fork, he will get the priority of the forks.
  - Nothing can then prevent him from taking the other fork and start eating.
  - When he returns the two forks, the next one can do the same.



Because:

- Each shared resource is assigned the priority of the highest priority task that will use it.
  - When a task is allowed to access a shared resource, it will inherit its priority.
- A task got the priority max of its own static and the ceiling priority of any resources it has locked
- A task can only lock a resource if the task's priority is **higher than the ceiling** of any currently locked resource (excluding any that it has already locked itself).

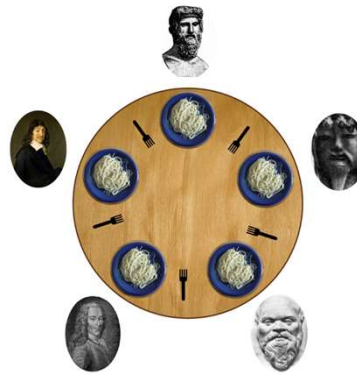
34

TTK4147 – 2025: Priority Inversion



### The dining philosophers' problem, some ending remarks

- Only one philosopher has been eating a time
- Have used a single CPU as view, sufficient with one eating philosopher a time.
- Have prevented dead locks, avoided circular wait.
- With a multi CPU / core would have used other approaches.
- By using ICPP and OCPP, the function of increasing the task's (philosophers') or resource's (fork's) priority to prevent interruptions when picking forks is used.



35

TTK4147 – 2025: Priority Inversion

NTNU

35

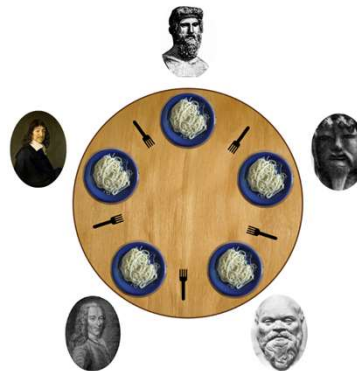
### The dining philosophers' problem, some ending remarks continue

In lab "Exercise 4 Concurrency" you also deal with the dining philosophers' problem on possibly more cores, with a focus on using semaphores.

Possible approaches:

- Only legal to grab a fork if none of your adjacent philosophers are holding a fork:  
Use an extra semaphore to protect the area of taking two forks (semaphores), and back out and wait if one or both forks is not free.

(Only possible with maximum two eating philosopher a time.  
5 forks, need 2 forks for each eating philosopher )



36

TTK4147 – 2025: Priority Inversion

NTNU

36



Also use

Blackboard → TTK4147 Sanntidssystemer (2025 HØST) →  
Course Information → Questions

37

TTK4147 – 2025: Priority Inversion

NTNU

37

## Endings

2018-14

Explain how the immediate or original ceiling protocols prevent deadlocks from occurring.

Each shared resource is assigned the priority of the highest priority task that will use it. A task inherit the priority of the resource it will use.

A task can only lock a resource if the task's priority is **higher than** the ceiling of any **currently locked resource** (excluding any that it has already locked itself).

- This means that when a task starts using a resource, it is guaranteed to be uninterrupted until it is done with the resource.

2018-15

If the **ceiling protocols** can prevent deadlock, explain why it is rarely (if ever) built-in to operating systems, while **priority inheritance** is almost universally present.

**Ceiling:** It is difficult for an OS to know in advance which task (and by that the priority) will ask for a resource, and by that decide the priority of a resource.

**Inheritance:** A blocked task gives its priority to the blocking task (if lower) → don't need information about the future.

38

TTK4147 – 2025: Priority Inversion

NTNU

38